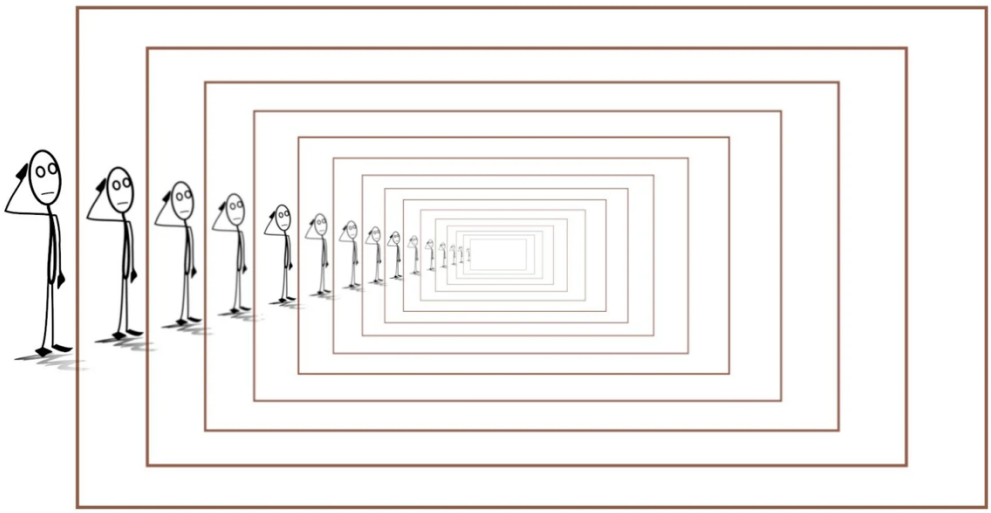
RECURSION



Recursion- Introduction

In Java, the function-call mechanism supports **the possibility of having a method call itself**. This functionality is known as recursion.

Recursion refers to the event when a function calls itself directly or indirectly. Recursion can be used at places where there is a certain relation between a sub-problem and the main problem, so sub-problems are solved individually, and solutions of all the sub-problems are combined to get the final result.

There are two main requirements of a recursive function:

- **A Stop Condition/Base Condition** – The function returns a value when a certain condition is satisfied, without a further recursive call.
- **The Recurrence Relation** – The function that calls itself with an input(parameter) which is a step closer to the stop condition. That function can be type void(don't return anything) so no need to store it or it can be of return type to any datatype so we need to store it.

Each recursive call will add a new frame to the stack memory of the JVM. So, if we don't pay attention to how deep our recursive call can dive, an out of memory exception may occur. Example Fibonacci Series of larger number with memoization concept.

Syntax for Recursion

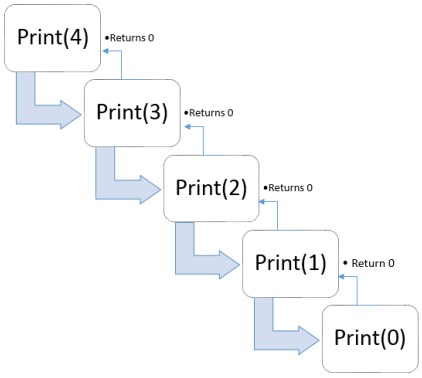
Every recursive code in programming must have a base condition that stops the recursion. If the condition is not provided or not given correctly it will lead to stack overflow error. There should be at least one recurrence relation. The java.lang.StackOverflowError indicates that the call stack memory is exhausted and is usually caused by deep or infinite recursion.

Check IDE for the syntax of recursive methods https://replit.com/@ManuChaitanya/recursion#Main.java

How Recursion Works

Let us suppose we a have to print till 1 to 10, so first we have to think about the base condition that should be something **(a==10)** then return. So how it works internally:

- First the main method is called hence it comes inside the stack.
- Then the function with **value-1(T1)** will come inside the stack. *As we know a function have only two task to do, either execute some LOC or call other method*. The function will first print 1 and will call same function with **value-2(T2)** and so on. The function with value 1 will wait until its work get completed, what do we mean by completion of work is to wait until the function that it has called - T(2) return its value to T(1).



- Now the function T(2) will come inside the stack and will wait until its work get completed.
- After T(10) is called by T(9) it will return after printing 10 from the base condition- completing its work to place where it was called- T(9). And then T(10) will come out of stack. The function T(9) will complete its task now it will return the value to the place where it was called- T(8) and will come out of the stack and so on.
- Finally T(1) will return the value from where it was called- Main Method and finally come out of stack. Then Main Method comes out of stack ending the programme.

Types of Recursion

There are two types of recursion based on when the recursive call is made to function.:

- Tail Recursion
- Head Recursion

Tail Recursion

When the last statement in the recursive method is the recursive call for that function, it is termed tail recursion. There is no further execution of any code after calling the recursive call. The recursive call is usually made along with the return statement.

Head Recursion

If the first statement in the recursive method is the recursive call, it is head recursion. *There's no statement, no operation before the call.* The function doesn't perform any operations at the time of calling, it will perform all the operational at the time of returning.

Stack Over Flow

Let us understand the stack overflow error from the example taken above, i.e., finding the sum up to 20. We have defined the base condition; if the value of i is equal to 1, then it returns 1, and hence recursion will break or else recursion continues. Check IDE to check the code.

After executing the program, it will display a stack overflow error as recursion will never reach the base case because we decrease the value of i by 1 every time. Hence call stack memory will get exhausted by these method calls, and an error occurs.

Recursion VS Iteration

Recursion can help to simplify the implementation of some complicated problems by making the code clearer and more readable.

But as we've already seen the recursive approach often requires more memory as the stack memory required increases with each recursive call.

As an alternative, if we can solve a problem with recursion, we can also solve it by iteration.

In comparison to recursion, the iterative approach could potentially give better performance. That being said, iteration will be more complicated and harder to understand compared to recursion, for example: traversing a binary tree.

Making the right choice between head recursion, tail recursion and an iterative approach all depend on the specific problem and situation.

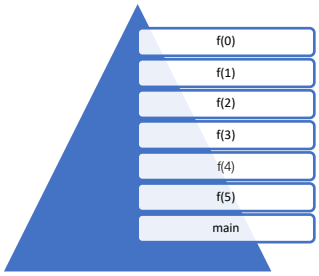
Recursion– Manu Chaitanya

Steps Recursion

Step-1: Thinking about what is that particular idea for which a big problem is being divided into sub problems. Like 5!, 5! =(5) * 4!, 4! = 4 * 3! and so on.

Step-2: Thinking about the parameters of the sub-problems, drawing Recursive Tree.

Step-3: Write the Recurrence Relation(n * (n-1)!) along with the base condition(n==0?return 1) and complete the Recursive Tree. Check where the method return the value, till the main method.

Step-4: Check the flow of programme and how it's working internally. How the Stack Memory is managed.

Step-5: Use debugger to check the flow of programme.